

Smart Task Management System — Architecture

A full-stack MERN application (TypeScript end-to-end) that lets users manage tasks with CRUD, search/filtering, dashboard analytics, and AI-assisted productivity features. It is built for individual users who want to capture tasks quickly and get AI-generated structure out of them — natural-language task parsing, productivity summaries, and ranked "what to do next" suggestions. The core value proposition is AI-assisted task management: the system turns free-text input into structured, prioritized tasks without removing human control over what actually gets saved.

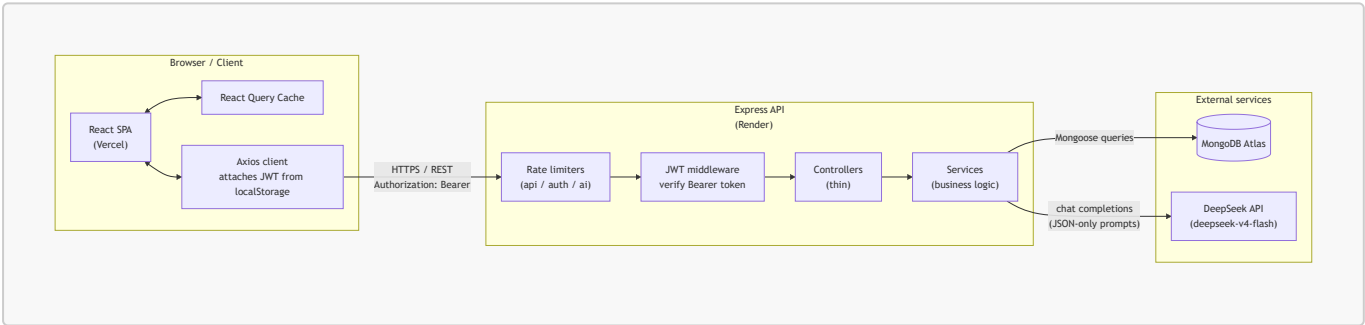
This document describes the system as implemented, based on the actual code in `client/` and `server/`.

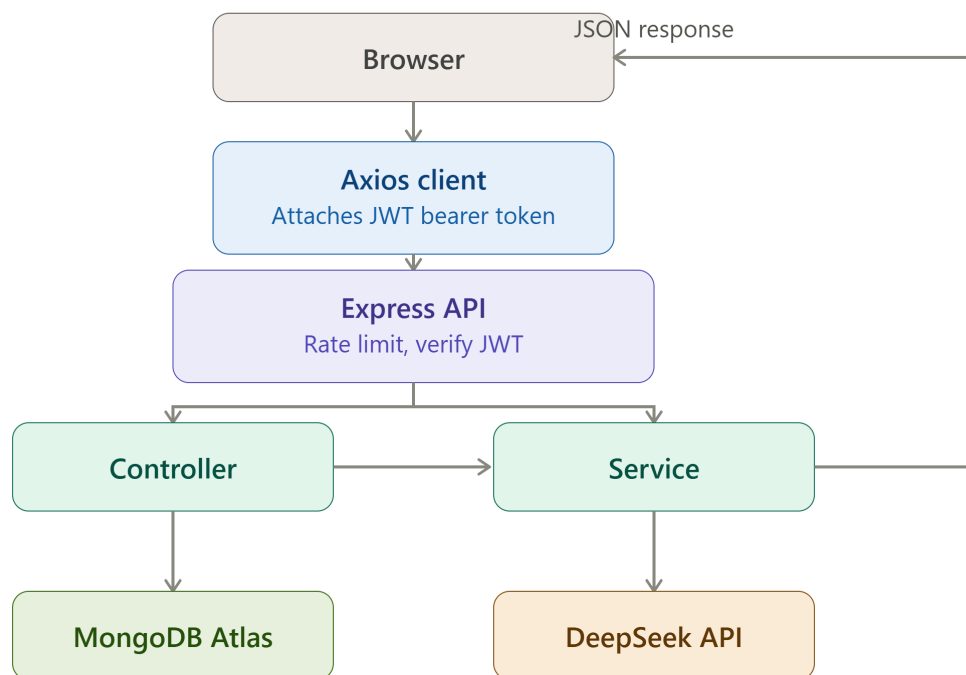
1. Tech Stack

Layer	Technology	Why chosen
Frontend framework	React 18 (<code>react</code> + <code>react-dom</code>)	Mature component model; ecosystem support; fits the SPA requirement
Build tool	Vite 5	Fast dev server and optimized production builds; first-class TypeScript support
Language	TypeScript 5.6 (both apps)	End-to-end type safety; shared shapes between API layer and features
Styling	Tailwind CSS 3.4	Utility-first styling with dark-mode support; no component-library lock-in
Server state	TanStack React Query 5	Automatic caching, deduplication, and invalidation of server data; eliminates hand-written loading/error plumbing
HTTP client	Axios 1.7	Request/response interceptors centralize JWT attachment and 401 handling
Routing	React Router 6 (<code>react-router-dom</code>)	Declarative routing with a <code>ProtectedRoute</code> wrapper for auth-gated pages
Backend framework	Express 4 (<code>express</code>)	Minimal, widely understood; the app's needs (REST + middleware) are simple enough that a heavier framework adds no value
Backend language	TypeScript (via <code>ts-node-dev</code> , compiled with <code>tsc</code>)	Same type system as the client; <code>tsc --noEmit</code> runs as a CI gate
Database / ODM	MongoDB Atlas + Mongoose 8	Schema definition and validation in one place; the <code>Task/User</code> models double as the source of truth for document shapes
Input validation	Zod 3	Runtime validation at the API boundary; schemas in <code>validators/</code> also export inferred types used by

Layer	Technology	Why chosen
		controllers and services
Auth	JWT (<code>jsonwebtoken</code>) + bcrypt (<code>bcryptjs</code>)	Stateless auth suits a decoupled SPA/API deployment; bcrypt hashes passwords at 10 rounds
AI	DeepSeek API (<code>deepseek-v4-flash</code>)	Cost-effective chat-completions endpoint called server-side only, so the API key never reaches the browser
Testing	Jest 29 + Supertest 7 + <code>mongodb-memory-server</code>	Integration tests exercise real routes against an in-memory MongoDB; the DeepSeek module is mocked, and a <code>fetch</code> spy asserts no real network calls
Deployment	Vercel (frontend) + Render (backend)	Free tiers for both; Vercel auto-deploys from <code>main</code> , Render hosts the API with env vars
Other hardening	<code>helmet</code> , <code>cors</code> (allow-list), <code>express-rate-limit</code>	Security headers, origin allow-list, and per-route rate limits (see §5)

2. System Architecture Diagram





End-to-end request flow: browser → Express API → MongoDB / DeepSeek → response

Auth flow (high level): on login/register the server returns a JWT (1h expiry) alongside the user. The client stores it in `localStorage` under the key `token` (`features/auth/context/AuthContext.tsx`). An Axios request interceptor (`api/axiosClient.ts`) attaches it as `Authorization: Bearer <token>` on every request. Server-side, `middleware/auth.middleware.ts` verifies the token with `jsonwebtoken`, attaches `req.user = { id }`, and rejects with 401 if missing, malformed, or expired. A client response interceptor clears the stored token and redirects to `/login` on any 401.

3. Project Structure

```

smart_task_manager/
├── client/                                # React SPA (Vite)
│   ├── src/
│   │   ├── api/                          # Axios instance + typed API wrappers
│   │   │   (axiosClient, authApi, taskApi, aiApi)
│   │   ├── components/
│   │   │   ├── layout/                  # App shell: Layout, Navbar, Sidebar,
│   │   │   │   DarkModeToggle, SidebarContext
│   │   │   └── ui/                      # Shared "dumb" primitives: Button, Input,
│   │   │       Modal, Badge, Spinner
│   │   ├── context/                    # ThemeContext – dark-mode UI state (server
│   │   │   state lives in React Query)
│   │   ├── features/                  # Feature-based modules; each owns its
│   │   │   components, hooks, and context
│   │   │   ├── auth/                  # AuthContext, Login/Register forms, useAuth
│   │   │   └── tasks/                 # Task CRUD UI: TaskList, TaskCard, TaskForm,
│   │   │       TaskFilters, types, useTasks, useTaskMutations
  
```

```

|   |   | ai/                                # AI UI: NLTaskInput, AISummaryCard,
AISuggestionList, useAIParse/useAISummary/useAISuggestions
|   |   | └─ dashboard/                      # StatsCards, Calendar, CategoryBreakdown,
UpcomingDeadlines, RecentActivity, TrendIndicator, QuickAdd, DueSoonBanner,
useTaskStats
|   |   |   | hooks/                          # Cross-cutting hooks (useDarkMode)
|   |   |   | pages/                          # Route-level pages: LoginPage,
DashboardPage, TasksPage, CalendarPage
|   |   |   | routes/                         # AppRoutes (route table) + ProtectedRoute
(auth gate)
|   |   |   | styles/                         # index.css – Tailwind base
|   |   |   | App.tsx                         # Provider tree: QueryClient → Theme → Auth →
Routes
|   |   |   | └─ main.tsx                     # Entry point
|   |   |   | └─ server/                      # Express API
|   |   |   |   | src/
|   |   |   |   | └─ app.ts                   # Express app: helmet, CORS allow-list, JSON
parsing, route mounting, error handlers
|   |   |   |   | └─ server.ts                 # Entry point: connectDB() then app.listen()
|   |   |   |   | └─ config/                  # env.ts (env parsing), db.ts (Mongoose
connect), deepseek.ts (DeepSeek client)
|   |   |   |   | └─ controllers/              # Thin request/response handlers: auth, task,
ai controllers
|   |   |   |   |   | middleware/              # auth.middleware (JWT verify), validate
(Zod), rateLimiter, error.middleware (notFound + errorHandler)
|   |   |   |   |   | └─ models/              # Mongoose models: User, Task (+ inferred
interfaces)
|   |   |   |   |   | └─ routes/              # Route tables: auth.routes, task.routes,
ai.routes
|   |   |   |   |   | └─ services/            # Business logic: auth.service, task.service,
ai.service (DeepSeek integration)
|   |   |   |   |   | └─ types/              # express.d.ts – augments Express Request
with `user`
|   |   |   |   |   |   | utils/              # asyncHandler, ApiResponse, AppError, logger
|   |   |   |   |   |   | └─ validators/      # Zod schemas: auth.schema, task.schema
(doubles as static types)
|   |   |   |   |   |   | └─ tests/
|   |   |   |   |   |   |   | integration/    # Supertest suites: auth.test, task.test,
ai.test + setup.ts (in-memory Mongo)

```

4. Key Architectural Decisions

Feature folders over type-based folders (client)

All code for one concern lives in one folder: `features/tasks/` holds the task UI components, its React Query hooks, and its types; `features/ai/` and `features/auth/` do the same. Only genuinely shared UI (`components/ui/`, `components/layout/`) and app plumbing (`api/`, `routes/`, `context/`) live outside features. This keeps related changes in one place (adding a task field touches one feature) and prevents the `components//hooks/` buckets from growing into unmanageable grab-bags as features are added.

React Query for server state

Task lists, stats, and AI results are all server state; React Query owns fetching, caching, loading/error presentation, and re-fetching. Mutations declare their invalidation explicitly — `useCreateTask` invalidates `["tasks"]` and `["tasks", "stats"]` on success, so the list and dashboard refresh without hand-written refetch calls. React Context is deliberately limited to auth session and theme (UI state), keeping it out of the data-fetching path.

JWT over session-based auth

The client and API are deployed on separate origins (Vercel and Render), so cookie-session auth would require cross-origin cookie configuration and CSRF defenses. A stateless JWT (signed with a server secret, 1h expiry) stored in `localStorage` and sent via `Authorization: Bearer` works cleanly across origins, requires no server-side session store, and is invalidated client-side on 401. The tradeoff (token revocation is not immediate) is acceptable for a personal task manager with a short token lifetime.

Service layer on the backend

Controllers are pure plumbing: extract `userId` from `req.user`, pass validated input to a service, wrap the result in the `ApiResponse` envelope (`utils/ApiResponse.ts`). All business logic lives in `services/`: task queries and stats aggregation, auth/password handling, and the entire DeepSeek integration. This makes the AI logic directly testable (the integration tests mock `callDeepSeek` at the module boundary), keeps route handlers free of logic, and centralizes where a future change would touch.

AI features: parse-then-confirm, never blind auto-apply

The AI integration follows a strict "suggest, don't decide" pattern:

- `POST /api/ai/parse` turns free text into a structured draft (`title`, `dueDate`, `priority`, `category`). The response is loaded into a review modal (`NLTaskInput.tsx` → "Review your AI-parsed task") — nothing is saved yet. Only when the user confirms does the client `POST /api/tasks`, flagging the task with `aiGenerated: true`.
- The server enforces the same caution at write time: `task.service.createTask` re-validates the AI-derived priority and falls back to server-side inference if the parse result is missing/invalid. Manual creates also receive server-inferred priority and tags, so the AI never controls the write path — it only informs it.
- All DeepSeek calls are made server-side through one wrapper (`config/deepseek.ts`), so the API key never ships to the browser, and every AI response is validated before it reaches the client (malformed output surfaces as a clean 502, not a crash).
- The two read-only AI features (summary, suggestions) are cached in-memory per user for 15 minutes to keep dashboard loads fast and avoid redundant paid calls.

5. Data Flow Example: "Add with AI"

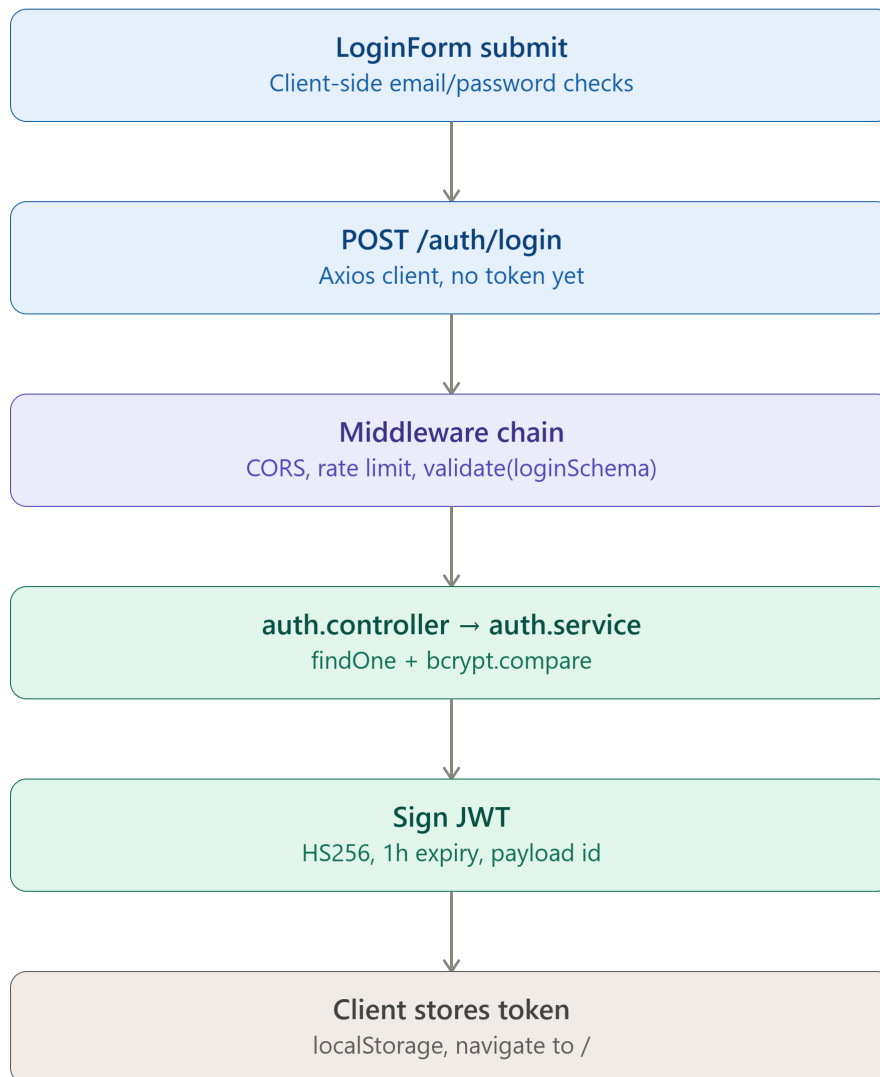
A concrete walkthrough of creating a task from natural language, using the real route names:

1. The user types "Submit report Friday at noon, high priority" into `NLTaskInput` (`features/ai/components/NLTaskInput.tsx`) and submits.

2. The `useAIParse` mutation (React Query) calls `aiApi.parseTask`, which POSTs to `POST /api/ai/parse` with `{ text }` and the JWT attached by the Axios interceptor.
 3. The server chain runs: `apiLimiter` (200 req / 15 min) → `authenticate` (JWT verify) → `aiLimiter` (30 req / 15 min) → `aiController.parseTask`.
 4. `aiService.parseTaskFromText` builds a system prompt (including today's date as the reference point for relative dates like "Friday") and calls `callDeepSeek`, which POSTs to `https://api.deepseek.com/chat/completions` (model `deepseek-v4-flash`, JSON response mode, 15s timeout).
 5. The service validates the returned JSON (title non-empty, priority in `low|medium|high`, `dueDate` a parseable ISO string) and returns the structured draft.
 6. The controller responds `{ success: true, data: { title, dueDate, priority, category } }`.
Nothing is written to the database.
 7. The client opens a modal titled "Review your AI-parsed task" with the fields pre-filled in `TaskForm`; the user can edit them, then confirms.
 8. Confirmation triggers `useCreateTask` → `taskApi.createTask` → `POST /api/tasks` with `aiGenerated: true`.
 9. The server validates the body with Zod (`createTaskSchema`), then `taskService.createTask` re-checks the AI-provided priority (only trusted when `aiGenerated: true`), infers tags server-side via `inferTags`, and writes the document with `TaskModel.create` — always scoped to `req.user.id`.
 10. `useCreateTask`'s `onSuccess` invalidates the `["tasks"]` and `["tasks", "stats"]` query keys; React Query refetches both, and the task list plus dashboard stats update immediately.
-

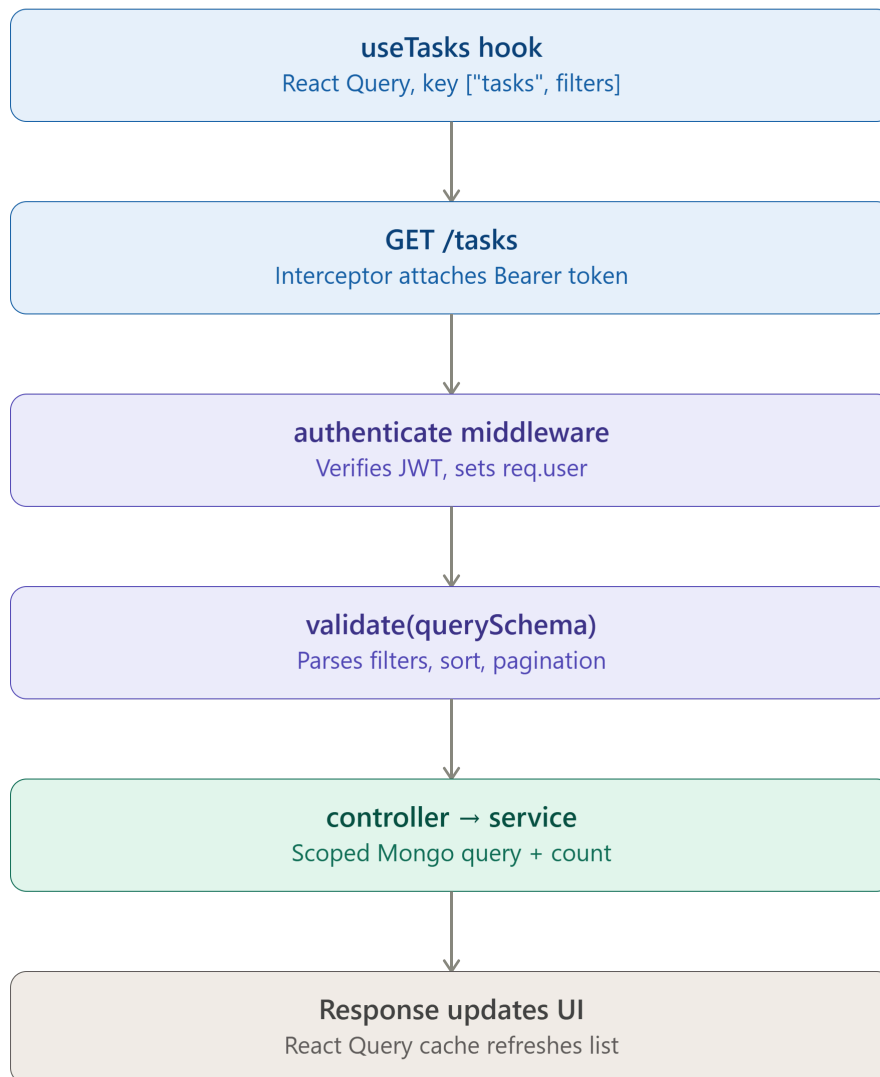
Detailed Request Flows

Login authentication flow



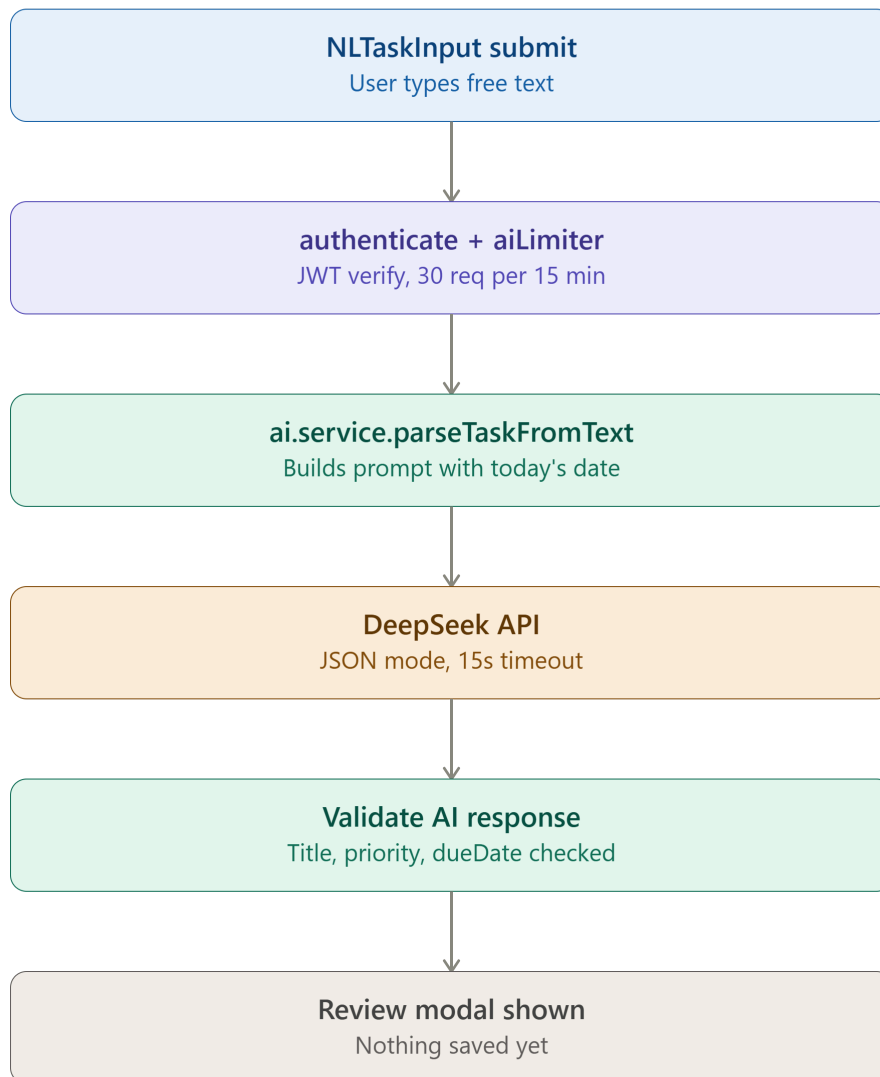
LoginForm's `handleSubmit` (`features/auth/components/LoginForm.tsx`) calls `AuthContext.login` (`features/auth/context/AuthContext.tsx`), which delegates to `authApi.login` — a POST to `/auth/login` through the shared `axiosClient`. The server chain runs `authLimiter` → `validate(loginSchema)` (Zod) → `authController.login` → `authService.login`, which executes `UserModel.findOne({ email }).select("+passwordHash")` (the field is `select: false` by default) and compares with `bcrypt.compare` before signing a JWT via `signToken` (HS256, 1h expiry, payload `{ id }`). The client stores the token in `localStorage` under the key `token`; the request interceptor in `api/axiosClient.ts` attaches it as `Authorization: Bearer <token>` on every subsequent request, and the response interceptor clears it and redirects to `/login` on any 401.

Protected request flow



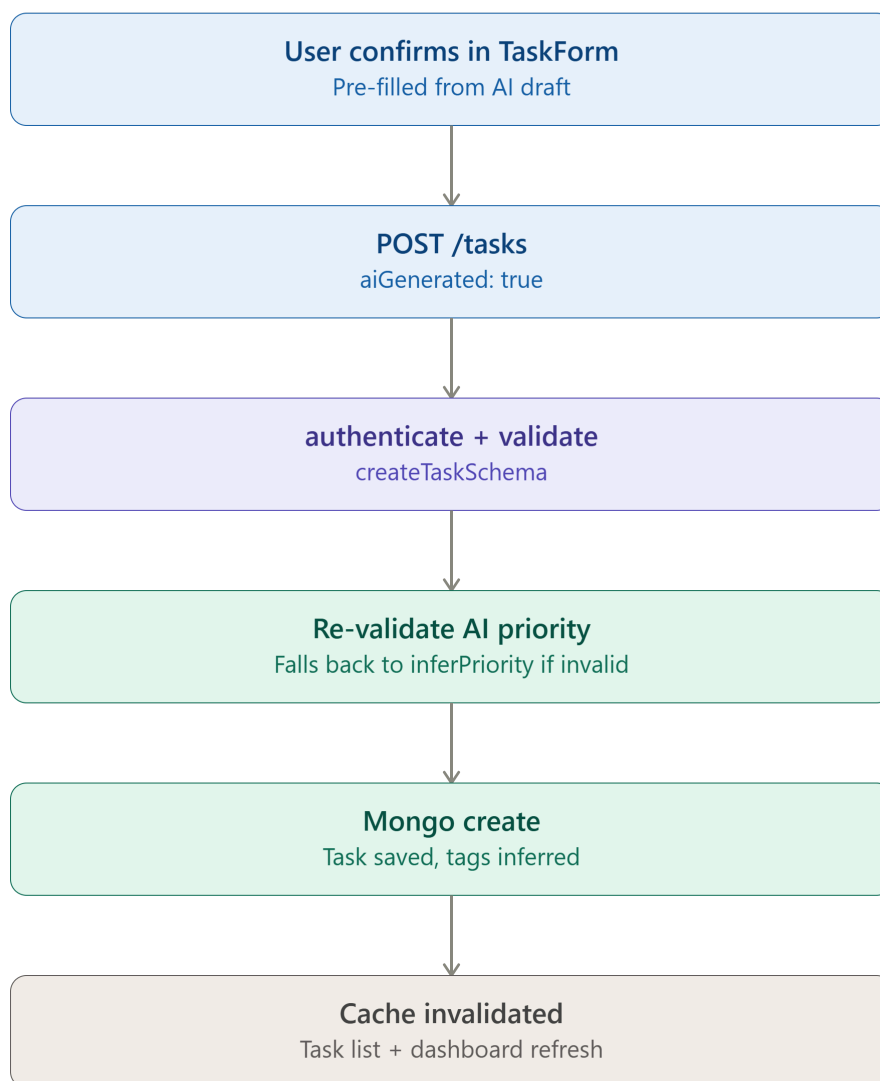
The request interceptor (`api/axiosClient.ts`) reads `localStorage["token"]` and attaches `Authorization: Bearer <token>`; any 401 response clears the token and hard-redirects to `/login`. On the server the chain is `apiLimiter → authenticate (middleware/auth.middleware.ts`, which runs `jwt.verify` and attaches `req.user = { id }`) `→ validate(querySchema, "query")` (Zod defaults: `sort=createdAt, order=desc, page=1, limit=10`) `→ taskController.listTasks → taskService.listTasks`. The service always scopes the filter to `userId`, adds optional `status/priority/category` equality, a case-insensitive `$or` regex on `title/description` for `search`, and a `dueDate $gte/$lte` range, then runs `TaskModel.find(...).sort().skip().limit().lean()` and `countDocuments` in parallel; every success is wrapped in the `ok()` envelope `{ success: true, data }` (`utils/ApiResponse.ts`).

AI parse flow



NLTaskInput's `handleSubmit` (`features/ai/components/NLTaskInput.tsx`) submits the free text through the `useAIParse` mutation → `aiApi.parseTask` → `POST /api/ai/parse`. Server-side the chain is `authenticate` → `aiLimiter` → `aiController.parseTask` → `aiService.parseTaskFromText`, which requires non-empty `text`, then calls `callDeepSeek` (`config/deepseek.ts`: `POST` to `https://api.deepseek.com/chat/completions`, model `deepseek-v4-flash`, `response_format: json_object`, 15s `AbortController` timeout). The service strictly validates the reply — non-empty `title`, priority within `low|medium|high`, `dueDate` a parseable ISO string — and throws `AppError(502)` for malformed output (never a 500). On success the client opens the "Review your AI-parsed task" modal with `TaskForm` pre-filled from the parsed draft; nothing has been written to the database yet.

AI confirm and create flow



Confirming in `TaskForm`'s `handleSubmit` builds the payload with the parsed fields, the reviewed priority, and `aiGenerated: true`, then runs `useCreateTask` → `taskApi.createTask` → `POST /api/tasks`. Server-side: `authenticate` → `validate(createTaskSchema)` → `taskController.createTask` → `taskService.createTask`, which re-validates the AI-provided priority with `normalizePriority` (only trusted when `aiGenerated === true`), falls back to server-side `inferPriority` when invalid or missing, and always infers tags via `inferTags` before `TaskModel.create` writes the document. The `201 ok(task)` response triggers `useCreateTask`'s `onSuccess`, which invalidates the `["tasks"]` and `["tasks", "stats"]` query keys so the task list and dashboard refetch.

Error handling matrix

Errors land in one of two places: middleware that responds **directly** (without `next(err)`) — `validate`, `authenticate`, the rate limiters, `notFound` — or `asyncHandler` (`utils/asyncHandler.ts`), whose `.catch(next)` forwards rejected promises to `errorHandler` (`middleware/error.middleware.ts`), which branches on error type in a fixed order: `AppError` → `ZodError` → `Mongoose ValidationError` → `Mongo 11000 duplicate key` → generic 500.

Failure case	Thrown at	Caught by	Resulting response
Invalid request body/query	<code>validate</code> middleware (<code>middleware/validate.ts</code>)	direct (returns before <code>next()</code>)	400 { error: "Validation failed", issues: [...] } }
Missing/malformed auth header	<code>authenticate</code> (<code>middleware/auth.middleware.ts</code>)	direct	401 { error: "Authorization header missing or malformed" } }
Invalid or expired token	<code>authenticate</code> — <code>jwt.verify</code> catch	direct	401 { error: "Invalid or expired token" } }
Missing <code>req.user</code> in controller	task/auth/ai controllers' <code>userId</code> guard	direct	401 { error: "Unauthorized" } }
Bad credentials	<code>authService.login</code> — <code>AppError(401)</code>	<code>asyncHandler</code> → <code>errorHandler</code> (<code>AppError</code> branch)	401 { error: "Invalid credentials" } }
Duplicate email on register	<code>authService.register</code> pre-check <code>AppError(409)</code> ; race condition surfaces as Mongo 11000	<code>errorHandler</code> <code>AppError</code> / 11000 branch	409 { error: "Email is already registered" } or 409 { error: 'Duplicate value for "email"' } }
Wrong owner / missing task	<code>taskService</code> — <code>assertValidTaskId</code> or null <code>findOne</code> result, <code>AppError(404)</code>	<code>asyncHandler</code> → <code>errorHandler</code>	404 { error: "Task not found" } }
Missing <code>text</code> on <code>/ai/parse</code>	<code>requireText</code> (<code>ai.service.ts</code>) — <code>AppError(400)</code>	<code>asyncHandler</code> → <code>errorHandler</code>	400 { error: "text is required" } }
DeepSeek timeout / network failure / non-200 / empty reply	<code>callDeepSeek</code> (<code>config/deepseek.ts</code>) — <code>AppError(502)</code>	<code>asyncHandler</code> → <code>errorHandler</code>	502 { error: "AI service unavailable: ..." } or 502 { error: "AI service returned status ..." } }
AI returned malformed/invalid	<code>aiService</code> (<code>ai.service.ts</code>) — <code>AppError(502)</code>	<code>asyncHandler</code> →	502 { error: "AI returned

Failure case	Thrown at	Caught by	Resulting response
JSON		<code>errorHandler</code>	<code>malformed JSON"</code> <code>}/ "AI returned</code> <code>an invalid task</code> <code>title" /</code> <code>"...invalid</code> <code>priority" /</code> <code>"...invalid</code> <code>dueDate"</code>
AI inference failure during create	<code>inferPriority</code> / <code>inferTags</code> try/catch (<code>ai.service.ts</code>)	swallowed by design	no error — falls back to <code>"Medium"</code> / <code>[]</code>
Mongoose validation failure	<code>TaskModel</code> / <code>UserModel</code> <code>ValidationError</code>	<code>errorHandler</code> (Mongoose branch)	<code>400 { error:</code> <code>"Validation</code> <code>failed", issues</code> <code>}</code>
Rate limit exceeded	<code>apiLimiter</code> / <code>authLimiter</code> / <code>aiLimiter</code> (<code>middleware/rateLimiter.ts</code>)	direct (express-rate-limit rejects)	<code>429</code> with <code>RateLimit-*</code> headers
Disallowed CORS origin	<code>cors</code> origin callback — <code>AppError(403)</code> (<code>app.ts</code>)	<code>errorHandler</code>	<code>403 { error:</code> <code>"Not allowed by</code> <code>CORS" }</code>
Unknown route	<code>notFound</code> (<code>middleware/error.middleware.ts</code>)	direct	<code>404 { error:</code> <code>"Not found" }</code>
Any unhandled error	anywhere in services/controllers	<code>errorHandler</code> fallback + <code>logger.error</code>	<code>500 { error:</code> <code>"Internal server</code> <code>error" }</code>

6. Testing Strategy

Backend integration tests run with Jest (`ts-jest`) + Supertest against a real Express app instance connected to an in-memory MongoDB (`mongodb-memory-server`, see `tests/integration/setup.ts`). Tests never touch the real database, and the DeepSeek module is mocked at the module boundary (`jest.mock("../src/config/deepseek")`) with an additional `fetch` spy asserting that no real network calls are made during the suite.

Current state: 79 tests passing across 3 suites (`npm test` in `server/`):

Suite	Coverage
<code>auth.test.ts</code>	Registration (duplicate email → 409, invalid payload → 400), login (generic 401 for wrong password and unknown email), <code>/me</code> with valid/missing/garbage tokens
<code>task.test.ts</code>	Create with all AI-priority edge cases (AI-provided, missing, invalid, inference fallback, failure fallback, tag inference), list with filters/search/date-range AND semantics,

Suite	Coverage
	pagination, ownership isolation (404, never 403, for another user's tasks), single-task GET/PUT/PATCH/DELETE, stats endpoint, auth guard
<code>ai.test.ts</code>	Parse/summary/suggestions from mocked DeepSeek responses, 15-min cache behavior, graceful 502 on malformed/AI failure (never 500), 401 guard on all AI routes, <code>inferTags</code> edge cases

The client has no test runner configured at present; its correctness is guarded by `tsc --noEmit` and ESLint, both wired into the package scripts and CI.

7. Deployment Architecture

The app is deployed in three parts:

- **Frontend — Vercel.** The React SPA is auto-deployed from `main`. The `VITE_API_URL` environment variable points at the Render backend's base URL (the Axios client falls back to `http://localhost:5000/api` in development).
- **Backend — Render.** The Express API runs as a web service. `CORS_ORIGIN` is set to the Vercel deployment domain; `app.ts` rejects requests from any origin not on that allow-list. `MONGO_URI`, `JWT_SECRET`, and `DEEPSEEK_API_KEY` are set in the Render dashboard and are never committed. A `GET /health` endpoint is available for uptime checks.
- **Database — MongoDB Atlas.** The M0 free cluster's network access allow-list is configured to accept connections from Render's egress IPs. `config/env.ts` requires `MONGO_URI` in production and fails fast if it is missing.

Free-tier caveat: Render's free web service spins down after inactivity, so the first request after an idle period incurs a cold start of roughly 30–60 seconds (the browser shows a loading state while the API boots and reconnects to MongoDB). Subsequent requests are fast until the next idle period.